IT 244 – Task 3: Architecture and System Requirements

ISU School Street Pantry AI Application | October 20, 2026

GitHub: https://github.com/jaybee1299/ISU-Pantry-Mobile-App

1) How request Move through the system

Every request goes through the browser and Flask. After that, the route decides what happens. Opening the home page loads the HTML template. Asking a question uses the embedding model and ChromaDB. Signing up uses SQLite instead. The route is predetermined, but which FAQ answer comes back depends on the question and the similarity search.

2) Request Workflow

User enters a question in the browser

↓

Flask /api/ask checks the question

↓

SentenceTransformer converts it to an embedding

↓

ChromaDB returns up to 3 FAQ matches

↓

Flask sends the best FAQ answer back to the browser

If the question is blank, Flask returns a 400 error. If no FAQ records are found, it returns a fallback message.

3) Application Architecture

Browser / HTML page	→	Flask (app/app.py)
AI question	→	SentenceTransformer + ChromaDB
Pantry / volunteer signup	→	SQLite signup database
FAQ preparation	→	CSV cleanup + vector DB build script

The AI uses cleaned pantry FAQ data stored in ChromaDB. SQLite is separate and stores signup information. The current AI selects a stored FAQ answer; it does not generate a new answer with a large language model.

4) Features

Must-have: a usable web page, pantry FAQ answers, pantry user and volunteer signup, basic form validation, and saved records. My top three nice-to-have features are (1) a relevance check for unrelated questions, (2) actual volunteer shift scheduling, and (3) an admin page to update FAQs and view signups.

5) Edge

Blank questions and missing signup fields already return errors. Other issues still need work: unrelated questions can receive a wrong FAQ match, email formats are not checked thoroughly, duplicate signups are possible, and database failures are not handled gracefully. The assistant also cannot confirm live pantry inventory because its FAQ data is static.

6) Evaluating the Answers

I would test the assistant with known FAQ questions, reworded questions, and unrelated questions. I would compare each result with the approved FAQ answer, check whether the correct record appears in the top three results, and count incorrect answers. I would also test signup errors and confirm that valid signups are saved. These tests are planned; I have not reported measured accuracy yet.

7) How to Clone and Run it

Clone the repository with Git, open the project folder, create a Python virtual environment, and run pip install -r requirements.txt. If the ChromaDB collection is missing, run scripts/build_vector_db.py. Then start the app with python app/app.py and open the local Flask address in a browser. The current model runs locally, so no paid AI API key is needed; the first run may download model files.

8) Deployment

I have not deployed this for public use yet. For a 5–15 person test, I would need hosting, HTTPS, a production server, and persistent storage for the databases.

9) Cost, reliability and Speed

Cost: the prototype uses free, open-source libraries; hosting would add a cost if deployed. Reliability: answers come from stored FAQ records, but the closest match is not always the correct match. Speed: AI requests include embedding and database search time, while signup requests mainly involve validation and SQLite. I still need to measure response times, so I cannot give a verified average or time-to-first-token. Since this version returns a stored answer rather than streaming generated text, total response time is the more useful metric.